

LR Using Regularization Methods - ST4035

Logistic Regression using Ridge and Lasso Regularizations

Let's use the PimaIndiansDiabetes dataset in the mlbench package.

```
#load required library  
library(mlbench)
```

Warning: package 'mlbench' was built under R version 4.4.3

```
#load Pima Indian Diabetes dataset  
data("PimaIndiansDiabetes")
```

Split the dataset into train and testing

```
#set seed to ensure reproducible results  
set.seed(42)  
#split into training and test sets  
PimaIndiansDiabetes[, "train"] = ifelse(  
  runif(nrow(PimaIndiansDiabetes)) < 0.8, 1, 0)
```

Separate training and test sets

```
trainset = PimaIndiansDiabetes[PimaIndiansDiabetes$train==1,]  
testset = PimaIndiansDiabetes[PimaIndiansDiabetes$train==0,]  
  
#get column index of train flag  
trainColNum = grep("train", names(trainset))
```

```
#remove train flag column from train and test sets
trainset = trainset[,-trainColNum]
testset = testset[,-trainColNum]
```

Build model

```
#get column index of response variable in dataset
typeColNum = grep("diabetes",names(PimaIndiansDiabetes))

#build model
glm_model <- glm(diabetes~.,data = trainset, family = binomial)
summary(glm_model)
```

Call:

```
glm(formula = diabetes ~ ., family = binomial, data = trainset)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-8.1485021	0.7835869	-10.399	< 2e-16 ***
pregnant	0.1200493	0.0355617	3.376	0.000736 ***
glucose	0.0348440	0.0040744	8.552	< 2e-16 ***
pressure	-0.0118977	0.0057685	-2.063	0.039158 *
triceps	0.0053380	0.0076523	0.698	0.485449
insulin	-0.0010892	0.0009789	-1.113	0.265872
mass	0.0775352	0.0161255	4.808	1.52e-06 ***
pedigree	1.2143139	0.3368454	3.605	0.000312 ***
age	0.0117270	0.0103418	1.134	0.256816

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 793.54 on 614 degrees of freedom
 Residual deviance: 584.57 on 606 degrees of freedom
 AIC: 602.57

Number of Fisher Scoring iterations: 5

Predict probabilities on test set

```
glm_prob = predict.glm(glm_model,  
                        testset[,-typeColNum],type="response")
```

Which classes do these probabilities refer to? What are 1 and 0?

```
contrasts(PimaIndiansDiabetes$diabetes)
```

```
      pos  
neg    0  
pos    1
```

Make predictions

```
#make predictions  
##.first create vector to hold predictions  
glm_predict = rep("neg",nrow(testset))  
glm_predict[glm_prob>.5] = "pos"  
  
#confusion matrix  
table(pred=glm_predict,true=testset$diabetes)
```

```
      true  
pred neg pos  
neg  90  22  
pos   8  33
```

```
#accuracy  
mean(glm_predict==testset$diabetes)
```

```
[1] 0.8039216
```

Ridge regularisation using glmnet

We use the glmnet package to fit the ridge model.

```
#load required library  
library(glmnet)
```

Warning: package 'glmnet' was built under R version 4.4.3

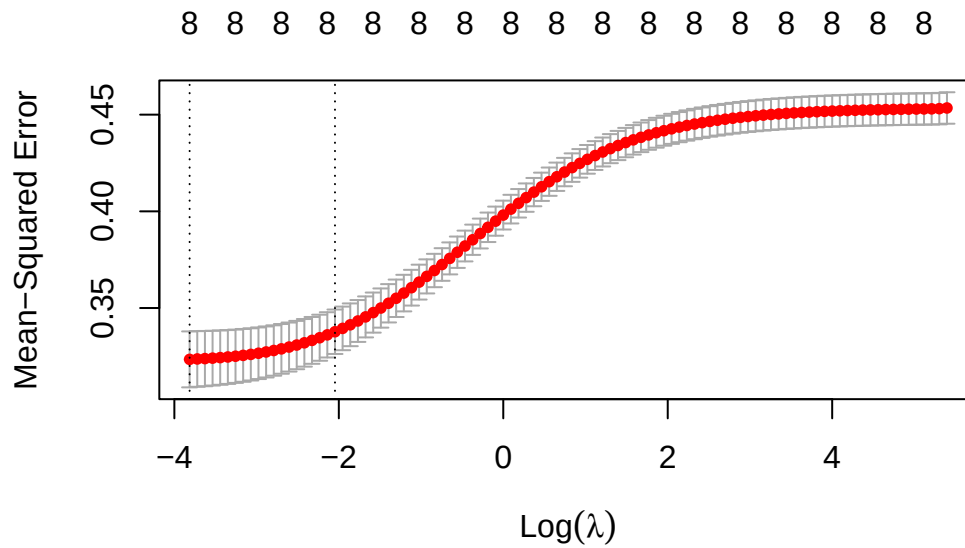
Loading required package: Matrix

Loaded glmnet 4.1-9

```
#convert training data to matrix format  
x = model.matrix(diabetes~.,trainset)  
  
#convert class to numerical variable  
y = ifelse(trainset$diabetes=="pos",1,0)
```

In order to fit a Logistic regression model using Ridge regularization, in the glmnet() function, family should be 'binomial' and alpha should be 0.

```
#family= binomial => logistic regression, alpha=0 => ridge  
cv.out.ridge = cv.glmnet(x,y,alpha=0,  
                        family="binomial",type.measure = "mse")  
  
#plot result  
plot(cv.out.ridge)
```



Using the cross-validation method, we can find the best λ value.

```
#min value of lambda
lambda_min.ridge = cv.out.ridge$lambda.min
#best value of lambda
lambda_1se.ridge = cv.out.ridge$lambda.1se
#regression coefficients
coef(cv.out.ridge, s=lambda_1se.ridge)
```

10 x 1 sparse Matrix of class "dgCMatrix"

```
      s0
(Intercept) -4.9654499180
(Intercept) .
pregnant    0.0613220816
glucose     0.0174051097
pressure    -0.0025726638
triceps     0.0031537210
insulin     0.0003154562
mass        0.0396309080
pedigree    0.6381233938
age         0.0130759197
```

Predictions for test data set

```

#get test data
x_test = model.matrix(diabetes~.,testset)
#predict class, type="class"
ridge_prob = predict(cv.out.ridge,
                     newx = x_test,
                     s=lambda_1se.ridge,
                     type="response")

#translate probabilities to predictions
ridge_predict = rep("neg",nrow(testset))
ridge_predict[ridge_prob>.5] = "pos"

#confusion matrix
table(pred=ridge_predict,true=testset$diabetes)

```

```

      true
pred neg pos
neg   92  25
pos    6  30

```

```

#accuracy
mean(ridge_predict==testset$diabetes)

```

```
[1] 0.7973856
```

Lasso regularisation using glmnet

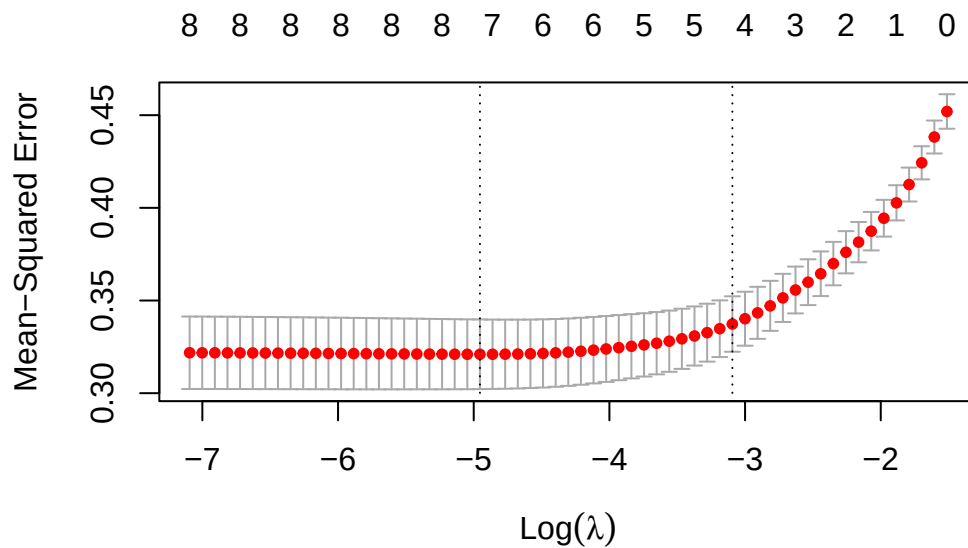
In order to fit a Logistic regression model using Lasso regularization, in the `glmnet()` function, family should be 'binomial' and alpha should be 1.

```

#family= binomial => logistic regression, alpha=1 => lasso
cv.out.lasso = cv.glmnet(x,y,alpha=1,
                        family="binomial",type.measure = "mse")

#plot result
plot(cv.out.lasso)

```



Using the cross-validation method, we can find the best λ value.

```
#min value of lambda
lambda_min.lasso = cv.out.lasso$lambda.min
#best value of lambda
lambda_1se.lasso = cv.out.lasso$lambda.1se
#regression coefficients
coef(cv.out.lasso, s=lambda_1se.lasso)
```

10 x 1 sparse Matrix of class "dgCMatrix"

```
      s0
(Intercept) -5.08324297
(Intercept) .
pregnant    0.04515256
glucose     0.02443349
pressure    .
triceps     .
insulin     .
mass        0.03346323
pedigree    0.33987321
age         .
```

Predictions for test data set

```
#get test data
x_test = model.matrix(diabetes~.,testset)
#predict class, type="class"
lasso_prob = predict(cv.out.lasso,
                     newx = x_test,
                     s=lambda_1se.lasso,
                     type="response")

#translate probabilities to predictions
lasso_predict = rep("neg",nrow(testset))
lasso_predict[lasso_prob>.5] = "pos"

#confusion matrix
table(pred=lasso_predict,true=testset$diabetes)
```

```
      true
pred neg pos
neg   93  25
pos    5  30
```

```
#accuracy
mean(lasso_predict==testset$diabetes)
```

```
[1] 0.8039216
```

Prediction comparison

```
df.print = cbind(mean(glm_predict==testset$diabetes),
                  mean(ridge_predict==testset$diabetes),
                  mean(lasso_predict==testset$diabetes))
rownames(df.print) = c("Accuracy")
colnames(df.print) = c("Logistic Regression","Ridge","Lasso")
df.print
```

	Logistic Regression	Ridge	Lasso
Accuracy	0.8039216	0.7973856	0.8039216